PROGRESS REVIEW

Failure-Aware Fast Rerouting in Software-Defined Networks

Interim progress: plan, emulator, failure detection and first results

Department of _____
Institute _____

Presented by

S. No.	Name of the Student	Roll Number
1.	Mayank Bansal	23BEC026
2.	Anas Khan	23BCS083
3.	Shardendu Mishra	23BCS122
4.	Himanshu Kumar	23BCS055
5.	Divyanshu	23BEC017

Under the guidance of

_____

(Name and designation of the guide)

Academic Year: _____ Date: _____

Problem and motivation

- In SDN, a failure is repaired by the **controller**: report → recompute → reprogram
- That takes **tens to hundreds of milliseconds**, and grows with controller distance
- OpenFlow **fast-failover groups** let a switch switch to a pre-installed backup locally
- But classic fast reroute treats every link as **equally reliable** and plans each backup **independently**
- Links in the same conduit fail **together**: a backup can die with its primary

Literature review

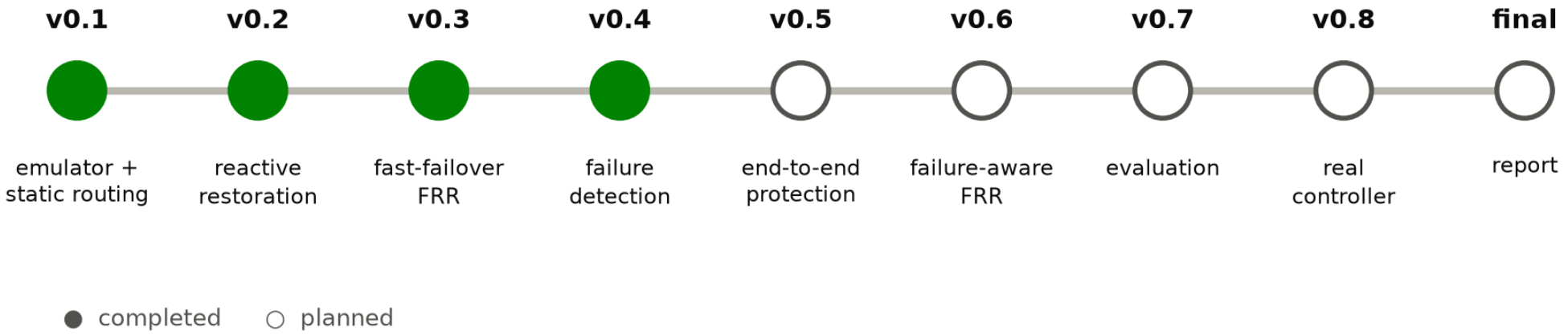
Work	Finding
Sharma et al. (2013)	only protection meets 50 ms recovery in OpenFlow networks
van Adrichem et al. (2014)	with fast-failover + BFD, detection dominates recovery time
IP LFA (RFC 5286), MPLS FRR (RFC 4090)	local link / node detours around the failure
Suurballe / Bhandari	disjoint path pairs for end-to-end protection
Gap	backups ignore failure likelihood and shared risk (SRLG)

Objectives

- Build a deterministic **OpenFlow emulator** (flow and group tables, port liveness, control-channel delay)
- Implement **baselines**: no recovery, reactive restoration, local fast reroute
- Model **failure detection** (LOS, BFD, LLDP) and gray failures and switch crashes
- Measure **outage, packet loss and switch state** on Abilene and NSFNET
- Design a **failure-aware fast reroute** scheme: risk model, SRLG-aware backups
- Port it to a **real controller** (OS-Ken/Ryu) for Mininet and Open vSwitch

METHODOLOGY

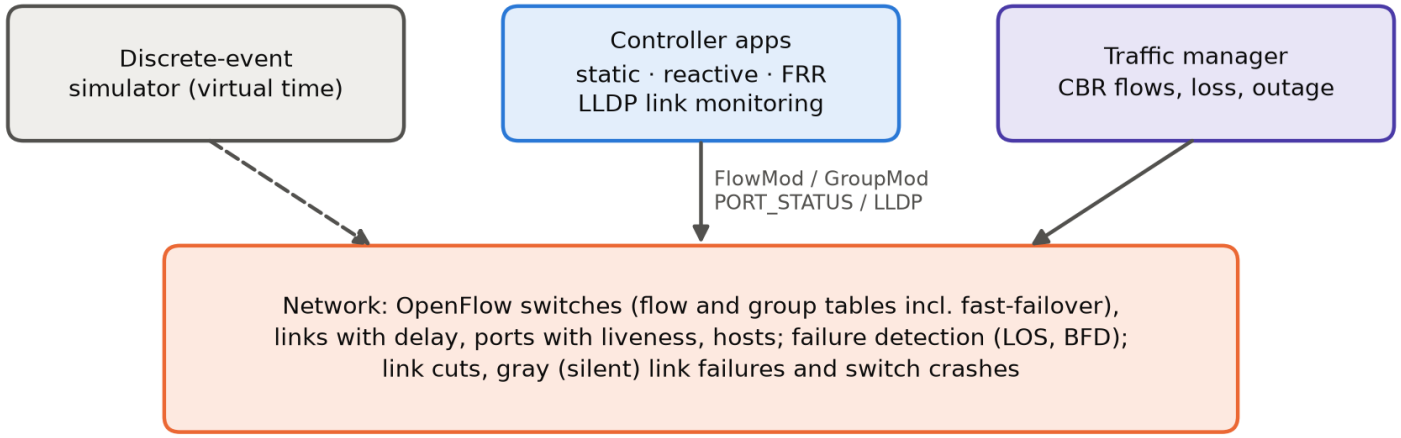
Plan: built in versions



Done: v0.1 emulator + static routing, v0.2 reactive restoration, v0.3 fast reroute, v0.4 failure detection. **Next:** end-to-end protection, then the failure-aware scheme.

IMPLEMENTATION

Work completed: the emulator



Control channel modelled with one-way delay, compute time and per-rule installation time.

- Python, ~2,100 lines
- **78 automated tests**
- Flow + group tables, incl. **fast-failover**
- Controller delay, compute and rule-install time
- Every packet sequence-numbered

IMPLEMENTATION

Strategies implemented

Strategy	How it recovers
static	shortest paths, no recovery (baseline)
reactive	controller recomputes on PORT_STATUS and pushes changed rules
FRR (link)	fast-failover group: primary + VLAN-tagged detour around the link
FRR (node)	detour around the whole next switch (also covers switch failures)

IMPLEMENTATION

Work completed: failure detection

Mechanism	Detects	Time	Cost
Loss of signal (switch)	link cuts only	~1 ms	free
BFD (switch)	cuts and gray failures	3 × hello interval	hellos on every link
LLDP (controller)	cuts and gray failures	3 × probe round + controller	probes via the controller

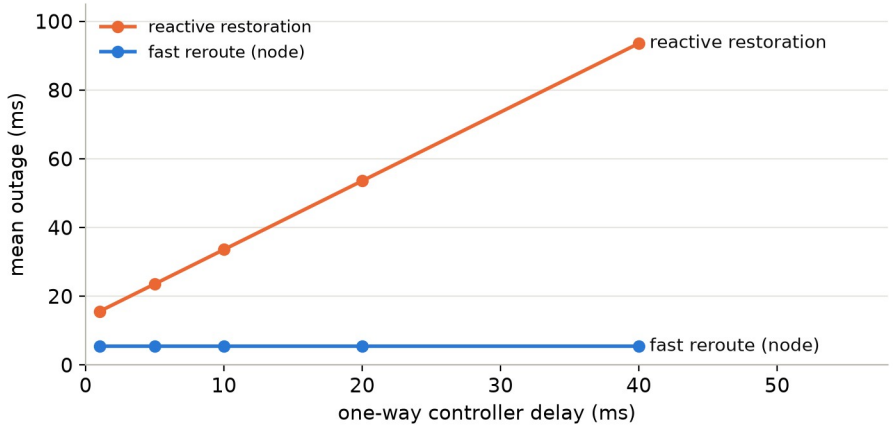
- Failure types: **link cut**, **gray (silent) failure** (signal up, traffic dropped), **switch crash**
- Every probe packet counted, so detection speed can be weighed against overhead

PRELIMINARY RESULTS

First results: single link failures

Strategy	Abilene	NSFNET	Flows / groups
static	never	never	121 / 0
reactive	23.3 ms	20.6 ms	121 / 0
FRR (link)	5.8 ms	3.0 ms	440 / 110
FRR (node)	5.8 ms	3.0 ms	404 / 110

Mean outage of affected flows, every link cut in turn



Fast reroute is **~4× faster** and independent of controller distance, at ~3.5× the flow entries.

PRELIMINARY RESULTS

First results: gray failures and switch crashes

Gray failure (DEN–KSC)	Outage	Probes
FRR, LOS only	never	0
FRR, BFD 3.3 ms	12.3 ms	5,937
FRR, BFD 10 ms	31.1 ms	1,960
FRR, BFD 50 ms	144.3 ms	392
reactive, LLDP 50 ms	167.6 ms	392

Crash of KSC	Never recovered	Outage
static	62 / 62	–
reactive	20 / 62	27.3 ms
FRR (link)	39 / 62	9.1 ms
FRR (node)	20 / 62	9.1 ms

Without BFD a gray failure is **never** noticed. On a crash, the 20 flows of KSC itself are lost for everyone; **node protection** recovers all the others in ~9 ms, link protection leaves 19 down.

PRELIMINARY RESULTS

The open problem: correlated failures

Strategy	Never recovered	Lost packets
static	50 / 50	10,549
reactive	0 / 50	1,339
FRR (link)	27 / 50	5,888
FRR (node)	22 / 50	4,887

- DEN–KSC and SNV–DEN share a conduit and fail together
- Backups for one use the other

Reactive recovers everything, slowly. Fast reroute is fast but leaves **22–27 flows dead**. A failure-aware scheme should be both fast and safe.

PLAN

Remaining work

Step	Content
v0.5	end-to-end path protection (disjoint paths)
v0.6	failure-aware FRR: risk model, SRLG-aware backups, re-optimisation
v0.7	evaluation: single, correlated, sequential and flapping failures
v0.8	OS-Ken/Ryu controller for Mininet and Open vSwitch
final	report and presentation

Thank you

Questions and feedback